

Development of an AI solution to improve public litter management

Vedant Vikramaditya*

Interlake Senior High School, Bellevue, WA, USA

February 2025

*vikramaditya.vedant@gmail.com

Contents

1	Introduction	1
1.1	Literature Review	2
1.1.1	AI Models	2
1.1.2	Datasets for Trash Classification	2
1.1.3	Real-World Applications	4
2	Methods	5
2.1	Hardware	6
2.2	Software	8
2.2.1	Raspberry Pi	8
2.2.2	Web Design	9
2.2.3	AI Model	10
3	Results	13
3.1	Object Detection	14
3.2	Bounding Box Detection	16
3.3	Inference Speed and Real-World Applicability	16
4	Discussion	17
4.1	Comparision	18
4.2	Further Research	19

Abstract

Litter pollution is a growing environmental issue, contributing to ecosystem damage, pollution apathy, and high cleanup costs. Traditional waste management methods rely on manual identification and collection, which are slow and resource-intensive, wasting person-hours and needing government spending. This project presents an AI-powered litter classification and tracking system designed for efficient waste management by government agencies and nonprofits. The system consists of three components: hardware, software (AI classifier and geolocation), and a web-based interface for dashboarding and analytics. The hardware is a modular, low-cost attachment that can be attached to any litter stick. This attachment captures images of collected waste and transmits them for analysis. The software consists of an AI model trained on large public datasets of >10,000 images and fine-tuned on around 4,000 self-collected images. From this training, it has classification times of <10 ms, making it ideal to run on edge devices. It is also capable of detecting trash with >93% accuracy. The hardware costs <\$30, allowing for mass distribution, including community give-aways. The collected data (classified trash and geolocation) are visualized through a web-based interface that allows organizations to track litter distribution, identify hotspots, and optimize cleanup efforts. By integrating AI and IoT technology, this system makes waste classification collection accessible and efficient, improving environmental cleanliness. The proposed solution provides a scalable and cost-effective method for tackling litter pollution, offering real-time insights to target waste management strategies. Through AI-driven automation, this project enables more efficient resource allocation, ultimately supporting environmental sustainability. sustainability.

1 Introduction

Litter has become a very big problem in the world and public litter management is usually insufficient and slow to respond to litter. Improper litter management has a significant impact on the environment, leading to pollution, ecosystem damage, and high cleanup costs. Litter can also contaminate outdoor environments, harming wildlife and plants. As population density increases in many areas, local waste management systems struggle to efficiently collect this trash. These efforts often rely on manual identification and tagging, which are slow and resource intensive. Advances in AI offer a potential solution by automating classification and tracking of litter pollution.

This research focuses on developing an AI model to classify trash in outdoor environments. By accurately identifying and categorizing waste, the model aims to assist government agencies and nonprofits in optimizing cleanup efforts and tracking litter patterns. Through this system, organizations can improve waste management strategies, reduce costs,

and implement more effective preventive measures to reduce littering. This study explores the feasibility of AI-driven waste classification and its potential to improve environmental sustainability and increase the efficiency of waste management organizations.

1.1 Literature Review

In recent years, artificial intelligence (AI) has emerged as a potential solution for automating waste classification and improving efficiency in cleanup efforts. This literature review explores previous research on AI-driven waste classification projects, focusing on machine learning models, data, and real-world applications. It examines existing methodologies, their effectiveness, and the challenges faced in developing accurate classification systems. Additionally, this review identifies gaps in current research, particularly regarding dataset diversity and real-world deployment. By analyzing prior work, this study aims to analyze the use of AI-based trash classification as a viable tool for government agencies and nonprofits to enhance waste management and environmental sustainability. This literature review is split into 3 parts: AI models, datasets for training, and real-world applications.

1.1.1 AI Models

Artificial intelligence has increasingly been used to automate waste classification, reducing the reliance on manual sorting and improving efficiency. The most common underlying model used for this is convolution neural networks (CNN). For example, in a paper by Shanshang Meng et al., a CNN with residual blocks is used for trash recognition for recycling aid [2]. One project that is meant for city-wide litter detection uses a light CNN with fewer layers [9]. This project boasts faster and less computationally-intensive detections than similar projects. Another project that is important for this research is the Trash Annotations in Context (TACO) project. The researchers behind the AI model associated with TACO use Mask R-CNN as the underlying model for their project, which is a variation of a CNN [4]. The TACO project’s model uses a Mask R-CNN, which allows it to create a pixel-by-pixel segmentation mask of multiple pieces of litter in a single image. This allows for more specific output with more information about the location and size of the piece of trash. This data about size is useful when working with local governmental agencies.

1.1.2 Datasets for Trash Classification

The availability of high-quality datasets is crucial for training AI models to classify waste accurately.

Several publicly available datasets, such as the Trash Annotations in Context (TACO) dataset, provide labeled and annotated images of litter in real-world environments [4]. These datasets help AI models generalize to different types of waste found in natural settings.



Figure 1: An example of an annotated and classified image from the TACO dataset

The TACO dataset also has a variation of environments where pictures were taken. As seen in Figure 2, TACO has a variety of environments, each with a large portion of images in it.

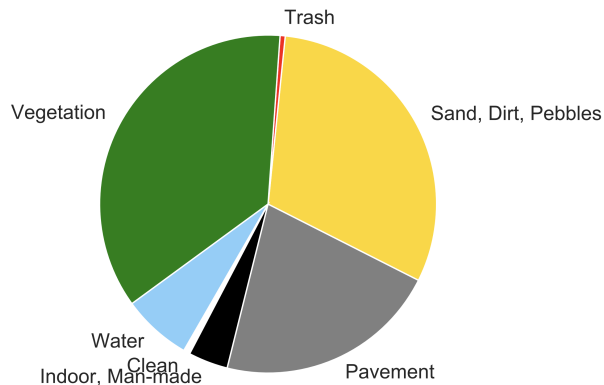


Figure 2: The distribution of environments based on image tags in the TACO dataset

While the TACO dataset provides high-quality images with diverse environmental contexts, it is limited to approximately 1,500 annotated images. In comparison, datasets like WasteNet offer over 10,000 images, but utilize clean, isolated images devoid of realistic backgrounds [9]. This lack of environmental context, as seen in Figure 3, can impact model generalizability.

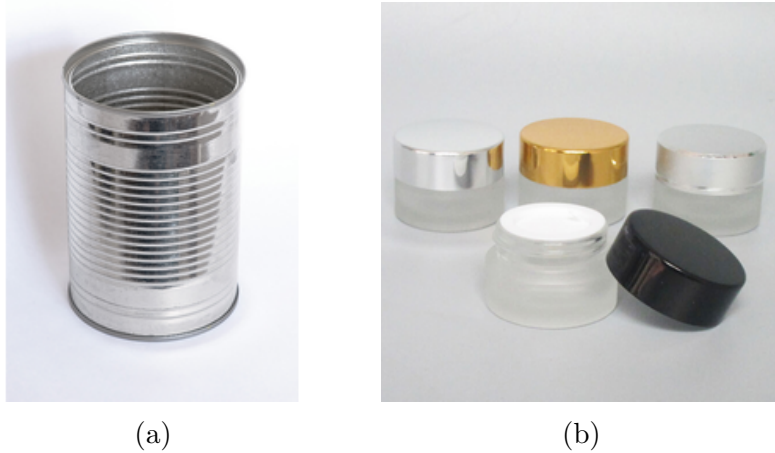


Figure 3: Example images from the WasteNet dataset

Because WasteNet’s data is not specific to the task the model needs to accomplish, it negatively impacts accuracy and limits the potential for high model performance [8]. While these datasets offer a large quantity of training data, they do not accurately represent the complexity of outdoor waste detection, where objects are often partially buried, crumpled, or obscured by other debris. This reliance on non-wild datasets presents a significant limitation in AI-based waste classification.

1.1.3 Real-World Applications

Several implementations of litter detection and management have been proposed for cities with extensive Internet of Things (IoT) infrastructure. These cities, known as smart cities, are defined by their integration of Information and Communication Technology (ICT) and IoT technology [7]. In [6], a simplified cost-benefit analysis was conducted on the implementation of smart waste management in a small urban area. This analysis began by utilizing an existing model and adapting it for urban littering scenarios, training it on images of active littering in public areas, as shown in Figure 4.



Figure 4: Images from the training dataset for [6]

It found that the smart waste management solution would save the city around £22,760 per year [6]. However, this analysis failed to address the cost of mischaracterization of trash and the suboptimal accuracy of the model. One of the most recent models aimed to tackle city-wide littering is LitterNet. The authors used a light CNN designed specifically for detecting littered face masks in urban environments. The model was trained using their custom dataset, MaskNet, which includes images of discarded face masks in various conditions such as partially hidden, crumpled, and under different lighting conditions. MaskNet achieves an impressive 96% accuracy in detecting discarded masks, outperforming traditional object detection models [3]. This project works very well in urban settings and environments, making it ideal for city use. It has also implemented mapping and interfaces to understand the data. This project is the closest to a successful project that can be used to inform city-wide litter management efforts. However, it is only specific to masks, making it not as useful as a general model.

2 Methods

To make this system practical and easy to use, it needs to work with existing infrastructure and be built with hardware, software, and a web-based interface. The hardware consists of camera-based sensors mounted on the litter pickup tools used by volunteers from local nonprofits and government organizations. These sensors need to be weather-resistant and capable of transmitting data reliably while remaining lightweight and unobtrusive.

The software component has two parts. First, is the interface between all the hardware to allow for a picture to be taken. Second, is the AI model which is responsible for identifying and categorizing trash. This model is trained on a diverse dataset to recognize different types of waste in outdoor environments. To ensure it works efficiently, the system must be accurate and able to run on local machines to make it accessible to everyone. To make the

collected data useful, it must have tracking features. This allows nonprofits and government agencies to track problem areas, identify patterns, and plan targeted cleanup efforts. The final part of the system is a web-based interface, which makes all this information easy to access. Volunteers and organizations can use it to visualize litter distribution, analyze trends, and manage cleanup operations more effectively.

Because these three components—hardware, software, and interface—are essential to the project, this paper will be structured around them, with each section focusing on one part of the system.

2.1 Hardware

The goal of this system is to be mobile with already existing litter cleanup crews. Because of this, the hardware for this must be portable, cheap, and easy to attach to already existing cleanup tools. This constrains our design to being something that can attach to a litter stick. Because this project needs to be cheap and easily attachable, an Amazon litter collector is used, as shown in Figure 5.



Figure 5: The litter stick being used for the project

This project also needs to be designed as a modular attachment specifically for litter collection tools used by volunteers. The system must be lightweight, cost-effective, and easy to use so that it does not interfere with the volunteers’ cleanup efforts. Since volunteers often operate in various outdoor environments, the attachment must be durable, weather-resistant, and capable of functioning without requiring complex setup or maintenance.

A key requirement of the system is a sensor to detect when trash is picked up and estimate its size, allowing for better tracking of litter collection efforts. This data can be used to analyze waste patterns and optimize future cleanup initiatives. To meet these constraints while maintaining affordability, the Raspberry Pi Zero W was chosen as the processing unit due to its compact size, low power consumption, and built-in wireless connectivity, which enables seamless data transmission. The PiCam V2 was selected as the imaging component

because of its high-resolution capture, lightweight build, and ability to function in different lighting conditions, ensuring accurate identification of collected trash.

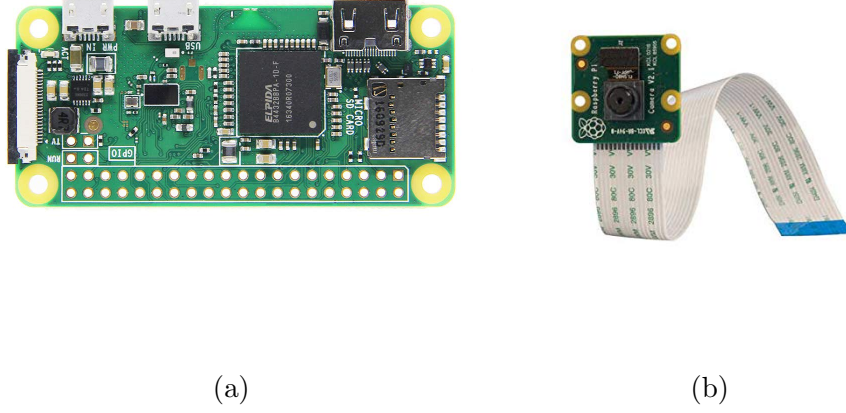


Figure 6: Off the shelf items used

The model of litter stick chosen for this project has attachment points built into the object. Because of this, the CAD uses a clip-on mount to hold the Raspberry Pi 0w and the PiCam V2. The clip-on mount for the Raspberry Pi is made to hold the Pi as close to the handle as possible to make sure that the weight doesn't make it harder to use the litter stick. The camera mount is placed nearer to the collection point to make sure that a high-quality picture is taken. The camera mount is angled downwards to make sure that the image of trash is taken in full view and is visible to the camera. There is also a battery mount placed using a generic battery near the top as well. This is all seen in Figure 7.

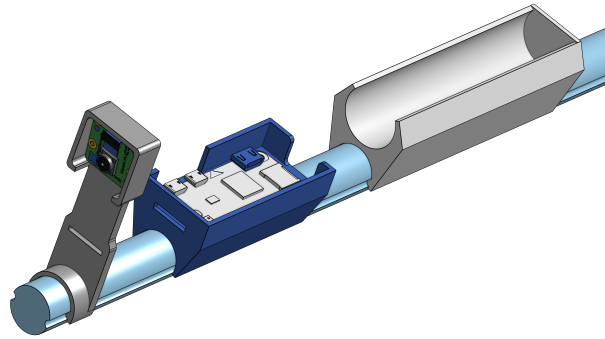


Figure 7: The designed mounts for the hardware

To get input from the user on when to take a picture, there is a button attached to the

handle on the manufactured version. There are also cables running between the Raspberry Pi, the camera, and the battery mount. There is also an off-the-shelf button attached to the litter stick to trigger a picture taken. Overall, the product costs less than \$30. In Figure 8, we see the button and wiring between all the objects.

2.2 Software

There are two sections to the software section. First, the software locally running on the Raspberry Pi will be discussed; and second, the AI model's development will be explained.

2.2.1 Raspberry Pi

Starting off with the code for the Raspberry Pi. The code needs to be simple, robust, and easy to understand for anyone. It also needs to interface with the button and the PiCam to take pictures and store them. The code running on the Raspberry Pi is designed to be lightweight and efficient, ensuring minimal delay between the button press and the image capture process. The program continuously monitors the button for input and, upon detection, triggers the camera to take a photo. This simple interaction ensures that volunteers do not need to focus on the technical aspects of the system, allowing them to focus on collecting litter. To accomplish this, the code utilizes Python with libraries such as **RPi.GPIO** for button input and **picamera2** for interfacing with the PiCam. When the button is pressed, the script activates the camera, takes a snapshot, and saves the image locally. This image is then transmitted to a central server or cloud platform where it will be processed by the AI model later. The image-saving process is straightforward, with each file being named with a timestamp to ensure proper sequencing and avoid overwriting. The code can be seen below:

```
1 import time ...
2 GPIO.setwarnings(False)
3 GPIO.setmode(GPIO.BOARD)
4 BUTTON_PIN = 7
5 GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
6 picam = Picamera2()
7 camera_config = picam.create_still_configuration()
8 picam.configure(camera_config)
9 picam.start()
10 previous_state = GPIO.HIGH
11 try:
12     while True:
```

```

13     current_state = GPIO.input(BUTTON_PIN)
14     if previous_state == GPIO.HIGH and current_state == GPIO.LOW:
15         timestamp = int(time.time())
16         name = f"/home/pi/Desktop/clipon/files/image_{timestamp}.jpg"
17         print(f"Capturing image: {name}")
18         picam.capture_file(name)
19     previous_state = current_state
20     time.sleep(0.01)
21
22 except KeyboardInterrupt:
23     print("Exiting program...")
24
25 finally:
26     picam.stop()
27     GPIO.cleanup()

```

This script is designed to be run continuously on the Raspberry Pi. The button is monitored, and when pressed, a picture is taken and saved with the current date and time. The image files are stored in a specified folder on the Raspberry Pi's filesystem, ready to be transmitted to the central server for analysis.

2.2.2 Web Design

The web design for this project was developed in Figma, a collaborative interface design tool that allows for efficient prototyping and iteration. Figma was chosen for its extensive features and developer-centered workflow, ensuring that the design was able to be executed quickly.

The design focuses on a clean, user-friendly interface optimized for accessibility and ease of navigation. It was also made to be as intuitive to understand all the data. The layout includes a dashboard for viewing a heat map of trash collected and real-time analytics. Components are structured with consistency in mind, using reusable design elements such as buttons, icons, and typography styles to maintain visual coherence.

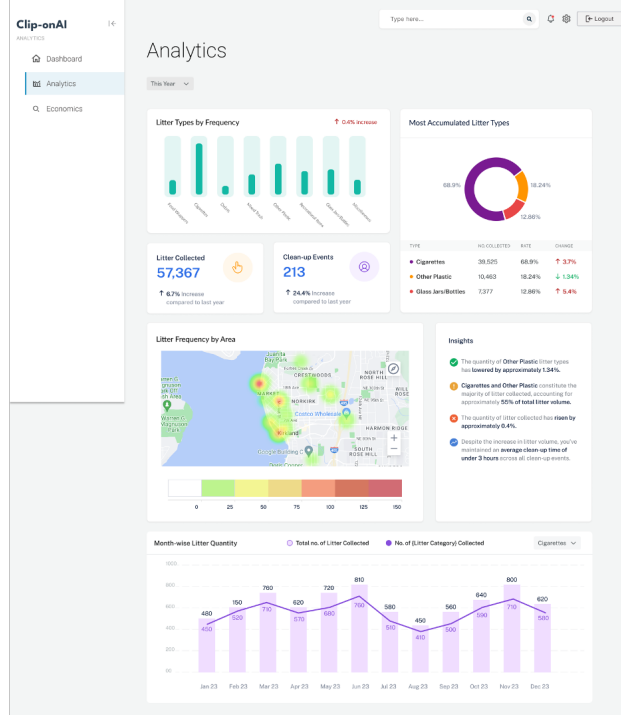


Figure 8: The Analytics section of the Website

Prototyping in Figma allows for testing interactions, ensuring intuitive workflows before implementation. The design also includes responsive elements to adapt to different screen sizes, making the web interface functional across both desktop and mobile devices. Once finalized, the design will be translated into code, maintaining fidelity to the original Figma layout while optimizing for performance and usability.

2.2.3 AI Model

The second key component of the software is the AI model used for classifying the litter in the images captured by the Pi Camera. This must be done with some type of an image classification model. Some of them were discussed in Section 1.1. At the start of this project, a couple of requirements for the AI model were made. First, it needs to be accurate. Second, it must be able to deal with an obstruction of the object being detected. Finally, it also needed to be simple to use.

First, the Mask R-CNN architecture was used. Mask R-CNN was previously discussed in 1.1 lightly in context with the TACO dataset and project. Because it has already been used, it had a lot of specific information about its use in relation to trash and litter classification. This makes it much easier to use than other models. Mask R-CNN is a type of deep learning model designed for instance segmentation. It extends Faster R-CNN by incorporating an

additional mask prediction branch, allowing it to not only detect objects but also generate pixel-wise segmentation masks for each instance.

At its core, Mask R-CNN relies on a convolutional neural network (CNN) backbone, typically ResNet, to extract features from an input image. CNNs work by applying convolutional filters across the image, capturing spatial hierarchies of patterns such as edges, textures, and shapes. Through successive convolutional layers, pooling operations, and activation functions, the network learns increasingly abstract features that enable it to recognize objects regardless of variations in lighting, orientation, and partial occlusion.

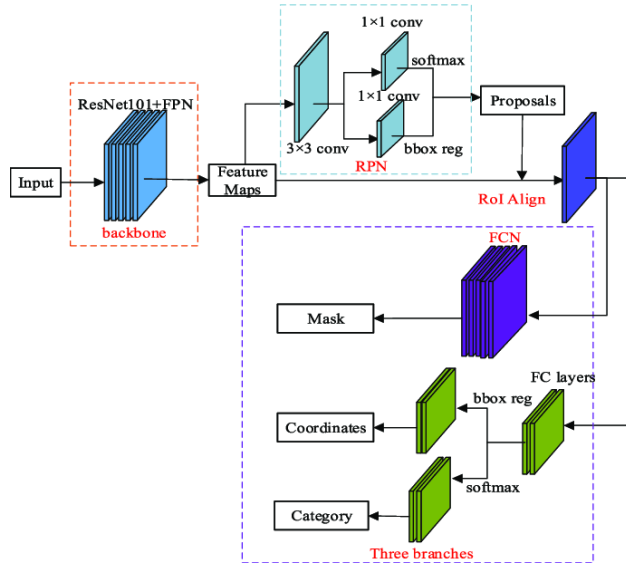


Figure 9: A flow-chart of Mask R-CNN

The key advantage of Mask R-CNN in handling occlusion comes from its pixel-wise segmentation capability. While a traditional object detection model will only provide a bounding box around an object, Mask R-CNN generates a binary mask that outlines the exact shape of the object. This allows it to differentiate between overlapping objects, as each instance is segmented separately rather than being enclosed within a single bounding box. Even if a piece of litter is partially obscured by another object, the segmentation mask can still identify the visible portions, making the model highly effective in complex, cluttered environments. Because of these advantages, this model was used first.

This model presented many problems, though. Starting off with the nature of Mask R-CNN. Mask R-CNN is a very complex model and its capabilities to do pixel-wise segmentation for multiple objects are very overkill for this project where a bounding box is sufficient. This means that Mask R-CNN needs many more steps and calculations to get a final result. It is around twice as slow as the model later used in inferences (YOLOv11) [5]. For inferences needed on edge devices that don't have high computational capacity, Mask

RCNN would be very slow, hurting the scalability of this project. Another problem was the complexity. It is an old architecture and runs on Python 2, making it very dated and hard to run on systems without many changes to the system itself. This made it very hard to run on other systems, making it impossible to scale. As well, Mask R-CNN is not very accurate. It has around a 25% worse accuracy than the model used in the end [5]. One of the biggest problems with this is the lack of training data. Since pixel-by-pixel segmentation masks are needed to annotate images for MASK R-CNN models, annotation is much more difficult to do for this model. Because of this, the only available data to train was from the TACO dataset which is just 1,000 images large. This is not enough to train a precise image classification model. From all these, new requirements were added to pick a useful model: it needs to be lightweight so that it can run quickly on hundreds or even thousands of images and it also needs to be easy to find and create annotations.

From these new requirements, YOLOv11 was chosen. To address Mask R-CNN’s inefficiencies, YOLO (You Only Look Once) was chosen for its speed and simplicity. Unlike Mask R-CNN, which requires multiple processing stages, YOLO predicts bounding boxes and class probabilities in a single forward pass, making it significantly faster and more efficient for edge devices. YOLO also handles occlusion well by leveraging a grid-based detection approach, predicting multiple bounding boxes per cell to detect partially visible objects. Non-Maximum Suppression (NMS) further refines detections by removing redundant predictions. Another advantage is its lightweight annotation process. Unlike Mask R-CNN, which requires pixel-wise segmentation masks, YOLO only needs bounding boxes, making dataset creation easier. Additionally, its widespread adoption provides access to extensive datasets, increasing data quantity. With twice the inference speed and higher accuracy than Mask R-CNN [5], YOLO was the ideal choice, meeting all key requirements: speed, ease of annotation, and scalability.

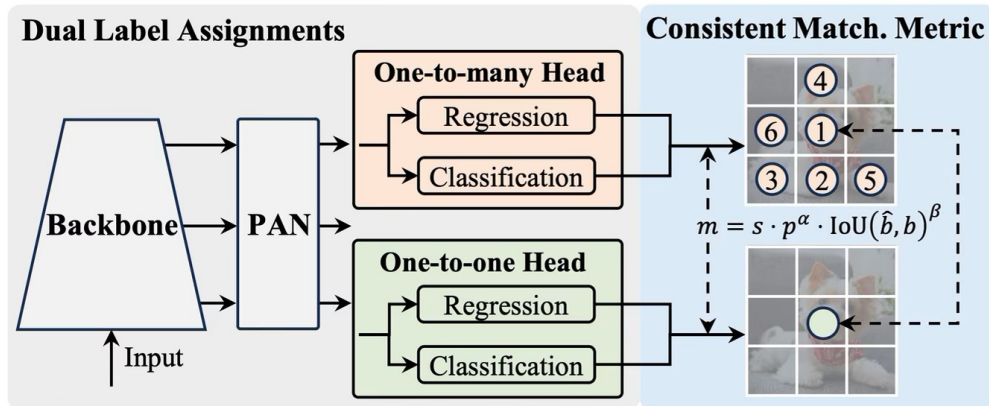


Figure 10: The architecture for YOLOv11

To make this model as accurate and precise as possible with the least parameters, a two-step training process was used. First, training was started with the online datasets discussed in Section 1.1.2, specifically WasteNet and TACO. This made the original model not very accurate, with an accuracy of around 65% on the validation images. This was a good start, but the model needed more specific data to be able to excel at its tasks. That is where the data that was specifically collected for this project comes in. Much of the data was taken from the POV of the litter stick camera, making it ideal for this specific fine-tuning. The outcomes and results of the AI model will be discussed in Section 3. In Figure 11, the dimensions and statistics about the self-collected data can be seen.

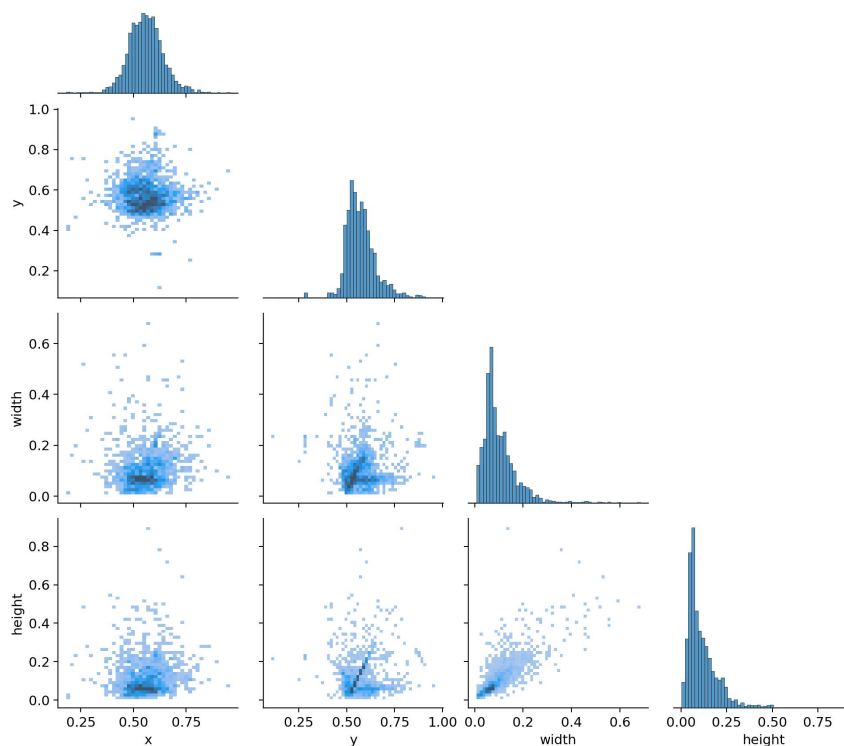


Figure 11: The correlogram for the self-collected data

3 Results

The AI model was evaluated based on many standard metrics to measure its quality and efficiency. The evaluation dataset consisted of just the self-collected images, because these were the highest quality data points that most nearly reflected the actual data that will be seen during use.

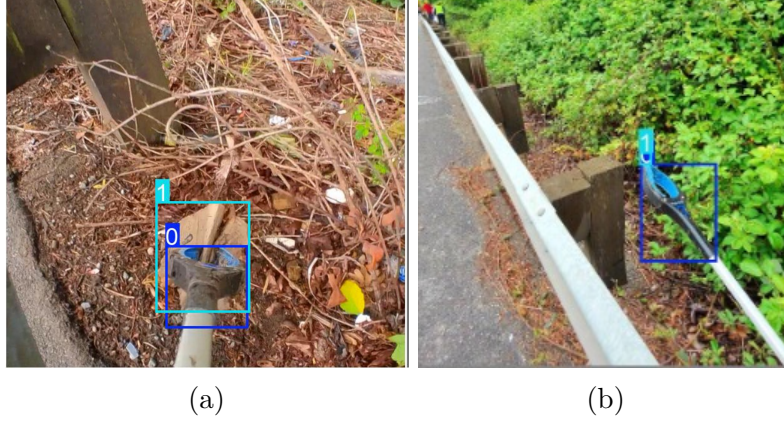


Figure 12: 2 examples of the model detection (1 is litter, 0 is grabber)

This model was also made to classify the trash by types. All of the possible classes for the trash are shown in Figure 13.

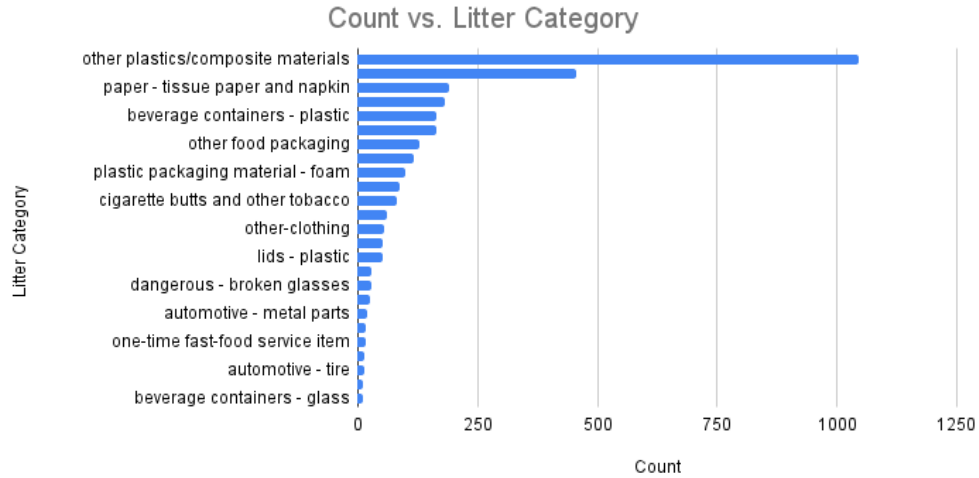


Figure 13: A bar chart of all the categories and # of objects in each

To ensure high accuracy, the litter categories were combined into one category of just "litter" to make sure that it is as accurate as possible.

3.1 Object Detection

The model has two ways to classify performance. It must be able to detect the object, and second, it must be able to create an accurate bounding box for the object to get its size. To quantify the accuracy of the model to detect objects, there are two metrics. First, the precision must be measured. The precision is how often the model is correct when it predicts

a positive result. The second way to quantify accuracy is recall. This is how often the model finds all the actual positive results in the image. First, calculating precision. This is very simple to calculate and is done using Equation 1.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}} \quad (1)$$

In Figure 14, the progression of the precision throughout the training process can be seen throughout the 75 epochs of training.

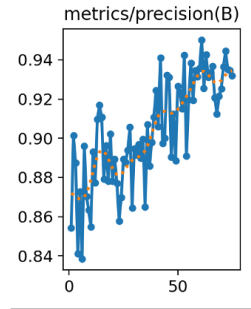


Figure 14: The graph of Precision vs. Epochs

Now, moving onto the recall of the model. Recall measures how often the model finds all the actual positive results. This is how well the model is at detecting the litter in the frame. This is calculated in Equation 2.

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}} \quad (2)$$

In Figure 15, the progression of the recall throughout the training process can be seen throughout the 75 epochs of training.

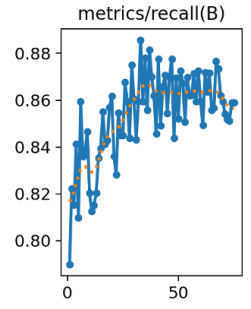


Figure 15: The graph of Precision vs. Epochs

3.2 Bounding Box Detection

Moving onto the second way to classify performance. This is about the bounding box detections. There are a couple different metrics to calculate this, but the most direct one is box loss. Box loss uses a metric called CIoU (Complete Intersection over Union). CIoU is a loss function used for bounding box regression that improves upon IoU by considering the overlap area, the distance between the center points of the boxes, and the aspect ratio consistency, leading to more accurate bounding box predictions. It directly addresses the limitations of IoU by providing better gradient information when boxes don't overlap, thus accelerating and stabilizing the training process. This is much more complicated to calculate and requires many more variables. This equation is seen in Equation 3. IoU is the Intersection over Union. b and b^{gt} are the center points of the predicted and ground truth boxes, respectively. $\rho^2(b, b^{gt})$ is the squared Euclidean distance between the center points. c is the diagonal length of the smallest enclosing box covering both boxes. α is a weighting factor. v is a term that measures the consistency of the aspect ratio.

$$\text{CIoU_loss} = 1 - \text{IoU} + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad (3)$$

In Figure 16, the progression of the box loss throughout the training process can be seen throughout the 75 epochs of training.

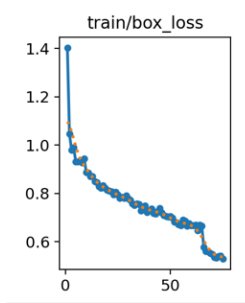


Figure 16: The graph of Box Loss vs. Epochs

3.3 Inference Speed and Real-World Applicability

Inference Speed: This model and the Mask R-CNN model were both benchmarked on an RTX 3080 AMD Ryzen 9 9950 x3d. The average inference time for the YOLO model was 8-10 ms. This means that the model can process around 110 images per second. As well, to make sure that this model can run on systems with lower specs, the model is trained to work on 640x480 images. This minimizes the memory needed to run to around 500 MB. Under partial occlusion, the model also performed well. This can be seen in Figure 17.



Figure 17: The graph of occluded object detection

These results validate the model’s effectiveness for automated litter classification, demonstrating its ability to accurately identify waste in diverse conditions. The system provides a scalable solution for environmental monitoring, allowing for widespread deployment across different regions. Additionally, its computational efficiency ensures that it can process data in real time, making it suitable for large-scale waste management applications.

4 Discussion

This study successfully developed and evaluated an AI model for automated litter detection, demonstrating promising performance across multiple metrics. The model, trained on a self-collected dataset of high-quality images, exhibited a robust ability to detect and localize litter objects, as evidenced by the convergence of loss functions and the improvement of precision and recall during training. The analysis of the training process, visualized through the graphs of precision, recall, and box loss, highlights the model’s effective learning and optimization over 75 epochs. Specifically, the box loss, which utilized CIoU to refine bounding box accuracy, showed a consistent decrease, indicating the model’s progressive ability to accurately localize objects.

4.1 Comparision

The model’s classification performance was evaluated by its ability to accurately detect objects and create precise bounding boxes. Precision and recall were analyzed to quantify detection accuracy, revealing a significant improvement over training epochs. Notably, the model achieved a precision of approximately 93% by the end of training. This high precision indicates that when the model detected an object, it was correct 93% of the time, surpassing the average human detection rate of around 70% as documented in [1]. This represents a 20% improvement in detection accuracy compared to human performance within the same timeframe, demonstrating the potential of AI to enhance litter detection efficiency. Furthermore, the model’s performance was benchmarked against the Mask R-CNN implementation by the TACO project, which reported a precision of 62.9%. This was calculated from the confusion matrix seen in Figure 18.

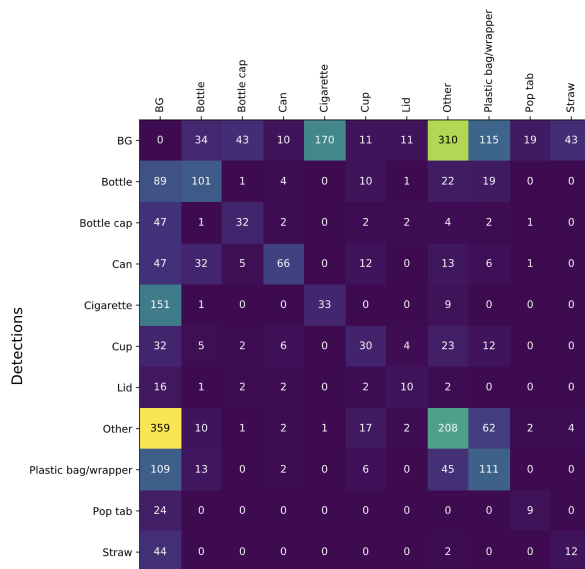


Figure 18: The confusion matrix for the TACO project

This model’s significantly higher precision underscores its superior performance in accurately identifying litter. Beyond the model’s performance, the project also emphasized the development of a user-friendly web interface and efficient hardware integration. The web interface was designed to provide easy-to-understand visualization of detection results, allowing users to monitor and interact with the system remotely. This interface is crucial for practical deployment, enabling easy access and control for waste management personnel. Furthermore, the model’s optimization for edge devices ensures that it can be deployed on a variety of hardware configurations, making it accessible for applications in diverse envi-

ronments. This focus on both software and hardware integration significantly enhances the project’s potential for real-world impact.

Even though this model is very accurate, it still makes some errors. When the background is very complicated, the model struggles to detect trash and can confuse the background for trash. Sometimes, when the litter is a similar color to the background, it is also unable to distinguish it. This is seen in Figure 19.



Figure 19: An example of a failed detection

In terms of inference speed, the model achieved an average processing time of 8-10 ms on an RTX 3080 and AMD Ryzen 9 9950 x3d, enabling the processing of approximately 110 images per second. This speed, coupled with the model’s ability to operate efficiently on lower-specification systems by processing 640x480 images with minimal memory usage (around 500MB), makes it suitable for real-time applications and edge device deployment. The model also demonstrated robustness under partial occlusion, as illustrated in Figure 17, further highlighting its practical applicability in diverse real-world scenarios.

4.2 Further Research

Looking forward, several avenues for future research and development can be explored. Firstly, expanding the training dataset to include a wider variety of environmental conditions and litter types could further enhance the model’s generalizability and robustness. Secondly, integrating the model with robotic systems or drones could automate litter collection processes, significantly reducing human effort and improving efficiency. Finally, exploring the

model’s performance on different edge devices and optimizing it for specific hardware configurations could facilitate its widespread deployment in real-world applications. An example of a possible heatmap for litter in Washington can be seen in Figure ??.

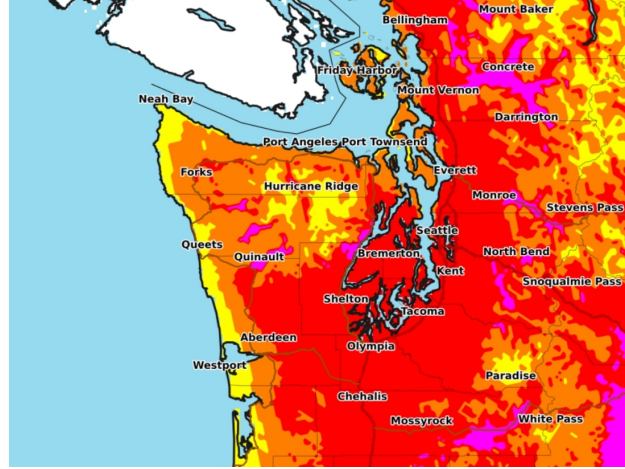


Figure 20: A possible litter heatmap in Washington

This model demonstrates a significant step forward in automated trash detection, offering the potential for real-world impact through its efficient deployment in smart city initiatives and environmental monitoring programs. By leveraging advancements in edge computing and AI, particularly with the optimized YOLOv11 architecture, the system enables rapid, on-site analysis, drastically reducing the time and resources required for litter detection and cleanup. Future development will focus on further enhancing its adaptability and accuracy, ultimately contributing to cleaner, more sustainable communities through widespread technological integration.

References

- [1] Jackelyn Hwang et al. “Curating Training Data for Reliable Large-Scale Visual Data Analysis: Lessons from Identifying Trash in Street View Imagery”. In: *Sociological Methods & Research* 52.3 (2023), pp. 1155–1200. DOI: 10.1177/00491241231171945. URL: <https://doi.org/10.1177/00491241231171945>.
- [2] Shanshan Meng and Wei-Ta Chu. “A Study of Garbage Classification with Convolutional Neural Networks”. In: *2020 Indo – Taiwan 2nd International Conference on Computing, Analytics and Networks (Indo-Taiwan ICAN)*. 2020, pp. 152–157. DOI: 10.1109/Indo-TaiwanICAN48429.2020.9181311.
- [3] Amir Peyvandi, Behzad Majidi, Sina Peyvandi, et al. “Location-aware hazardous litter management for smart emergency governance in urban eco-cyber-physical systems”. In: *Multimedia Tools and Applications* 81 (2022), pp. 22185–22214. DOI: 10.1007/s11042-021-11654-w.
- [4] Pedro F Proença and Pedro Simões. *TACO: Trash Annotations in Context for Litter Detection*. 2020. arXiv: 2003.06975 [cs.CV]. URL: <https://arxiv.org/abs/2003.06975>.
- [5] Ranjan Sapkota, Dawood Ahmed, and Manoj Karkee. “Comparing YOLOv11 and Mask R-CNN for instance segmentation in complex orchard environments”. In: *Artificial Intelligence in Agriculture* 13 (2024), pp. 84–99. ISSN: 2589-7217. DOI: <https://doi.org/10.1016/j.aiaa.2024.07.001>. URL: <https://www.sciencedirect.com/science/article/pii/S258972172400028X>.
- [6] Anusha Shenoy. “MSc in Business Analytics”. Master’s Thesis. Dublin Business School, 2024. URL: <https://hdl.handle.net/10788/4580>.
- [7] Dung D. Vu and Georges Kaddoum. “A waste city management system for smart cities applications”. In: *2017 Advances in Wireless and Optical Communications (RTUWO)*. 2017, pp. 225–229. DOI: 10.1109/RTUWO.2017.8228538.
- [8] Seung Eun Whang, Youngjin Roh, Hyunwoo Song, et al. “Data collection and quality challenges in deep learning: a data-centric AI perspective”. In: *The VLDB Journal* 32 (2023), pp. 791–813. DOI: 10.1007/s00778-022-00775-9.
- [9] Gary White et al. *WasteNet: Waste Classification at the Edge for Smart Bins*. 2020. arXiv: 2006.05873 [cs.CV]. URL: <https://arxiv.org/abs/2006.05873>.